

Redes de Computadores

Sumário

1	Introdução	3
1.1	O que é a Internet?	3
1.2	O que é um Protocolo ?	3
1.3	Borda da Rede	4
1.4	Núcleo da Rede	7
1.5	Performance	11
1.6	Camadas de Protocolos e Modelos de Serviços	14
2	Camada de Aplicação	16
2.1	Princípios da Camada de Rede	16
2.2	Web e HTTP	21
2.3	E-mail, SMTP, IMAP	26
2.4	DNS: Domain Name System	26
2.5	Streaming de Vídeo e Distribuição de Conteúdo na Internet	26
2.6	Programação de Socket com UDP e TCP	26

1 Introdução

1.1 O que é a Internet?

Bilhões de dispositivos se conectam à Internet atualmente.

- **Hosts (sistemas finais) :**
 - ↪ Exemplos: smartphones, notebooks, máquinas servidoras, etc.
 - ↪ Executam **aplicações de rede** e **interagem com o usuário**.
- **Packet switches (comutadores de pacotes) :**
 - ↪ Incluem **roteadores** e **switches**.
 - ↪ Responsáveis por transportar pedaços de dados (pacotes) de um ponto ao outro.
- **Communication links (enlaces de comunicação) :**
 - ↪ Recursos necessários para conectar dispositivos (finais, switches, roteadores, etc.).
 - ↪ Tipos :
 - **Cabeados:** fibra óptica, cobre.
 - **Sem fio:** Wi-Fi, satélite.
 - ↪ **Taxa de transmissão:** também chamada de Banda.
- **Redes :**
 - ↪ Coleção de dispositivos e roteadores sob uma gerência centralizada por uma instituição.
 - ↪ **Tipos:**
 - **Públicas:** padrões abertos, como HTTP, Ethernet.
 - **Proprietárias:** mantidas por instituições privadas, como o Skype.
- **IETF (Internet Engineering Task Force) :**
 - ↪ Responsável por gerenciar e controlar os padrões da Internet.
 - ↪ Cada padrão é lançado como um **RFC (Request for Comments)**.
 - ↪ Define e especifica as **características de um protocolo**.
- **Internet como infraestrutura de serviços :**
 - ↪ Provê serviços para **aplicações que executam em sistemas finais**.
 - ↪ Exemplos: web, streaming de vídeo, e-mail, jogos, redes sociais, etc.
- **Interface de programação para aplicações distribuídas :**
 - ↪ Permite que uma aplicação envie mensagens pela Internet.
 - ↪ Garante que o receptor consiga receber a mensagem de forma apropriada.

1.2 O que é um Protocolo ?

- Um **protocolo** define o formato, a ordem das mensagens enviadas e recebidas entre entidades da rede, e as **ações a serem tomadas** ao receber ou transmitir uma mensagem.

1.3 Borda da Rede

- Composta pelos **sistemas finais** (**hosts**: clientes e servidores, geralmente em *data centers*).
- A **conexão** desses dispositivos à rede é feita através das **redes de acesso** (*sem fio ou cabeadas*).
- O restante da rede é chamado de núcleo da rede — responsável pela interconexão de roteadores (a “rede de redes”).

Como conectar sistemas finais aos roteadores de borda ?

- **Redes de acesso residenciais**
- **Redes de acesso institucionais** (escolas, empresas)
- **Redes de acesso sem fio** (WiFi, 4G/5G)

Redes de Acesso

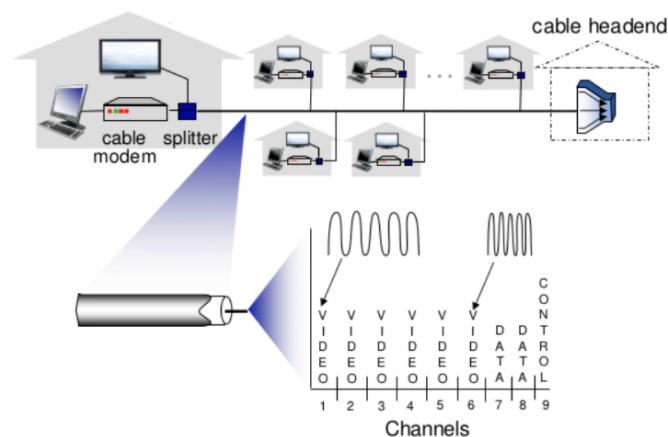
- **Taxa de transmissão (bits por segundo)** que o acesso permite — geralmente ligada à forma de compartilhamento do enlace e à capacidade de saída do roteador.
- O acesso é **dedicado** ou **compartilhado** entre os usuários ?

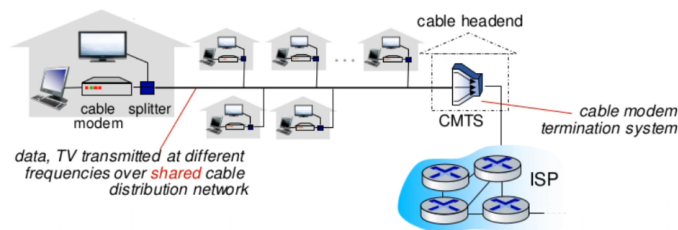
Exemplos de Redes de Acesso

TV a Cabo

- Utiliza o **divisor de frequência** ou **multiplexador FDM** (*Frequency Division Multiplexing*) :

↔ Diferentes canais de transmissão em diferentes faixas de frequência.

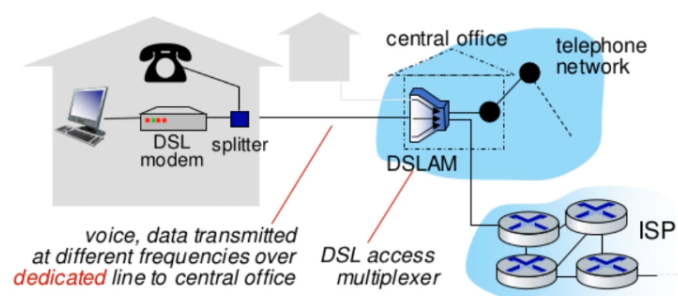




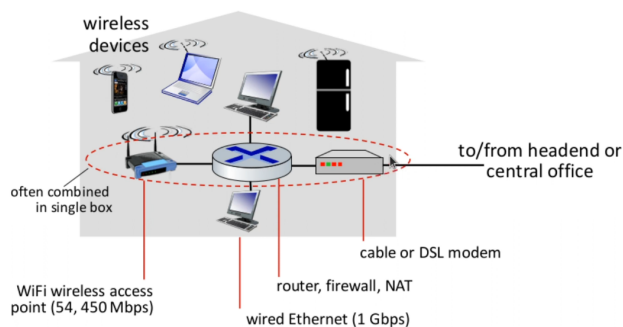
- Utiliza-se **HFC (Hybrid Fiber Coaxial)**, uma solução híbrida :
 - ↪ O **cabo coaxial** vai até o usuário.
 - ↪ A **interconexão** da rede é feita usando **fibra óptica**.
- **Assimétrica**: a capacidade de download e upload são diferentes.
- **Meio compartilhado**: o enlace é compartilhado entre vários usuários domésticos.

Rede baseada em DSL (*Digital Subscriber Line*)

- Utiliza os **cabos telefônicos** como infraestrutura de enlace.
- A conexão não é compartilhada.
- **Assimétrica** — download e upload possuem taxas diferentes.



Rede Doméstica

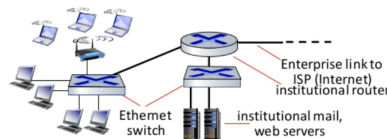


Redes de Acesso Sem Fio (Wireless)

- O **roteador de borda** é conectado a um **ponto de acesso**.
- **Locais**: alcance reduzido (domésticas, empresas) — **WiFi**.
- **Longas distâncias**: móveis (4G/5G), com pontos de acesso em **torres**, menor capacidade de transmissão.

Redes de Acesso Institucionais

- Utilizadas em **universidades, empresas, etc.**
- Maior complexidade de interconexão — acesso sem fio conectado a switches e roteadores.
- Tecnologias utilizadas :
 - ↪ **Ethernet**
 - ↪ **WiFi**



Hosts (Sistemas Finais)

- Quando o **host** tem conectividade, é possível enviar **pacotes de dados** até o destinatário.
- **Função de envio :**
 - ↪ Pega uma **mensagem** de uma aplicação.
 - ↪ Quebra essa mensagem em **pacotes menores** de tamanho L bits.
 - ↪ Esses pacotes são transmitidos a uma taxa de transmissão R (**bits/s**), correspondente à capacidade do enlace.
- Tempo de retardo (delay) = tempo necessário para transmitir um pacote:

$$\text{delay} = \frac{L(\text{bits})}{R(\text{bits/s})}$$

Links (Enlaces)

- Utilizados para a **interconexão de dispositivos**.
- Caracterizados por propriedades físicas.
- O **bit** é propagado entre transmissor e receptor.
- **Tipos de enlaces :**
 - ↪ **Guiados:** sinal se propaga em meio sólido (fibra óptica, cabos coaxiais, par trançado).
 - ↪ **Não guiados:** sinal se propaga de forma livre (ondas eletromagnéticas).

Cabos Par Trançado (Twisted Pair - TP)

- Categorias mais comuns: **Cat 5, Cat 6**.



Cabos Coaxiais

- Dois condutores de cobre concêntricos.
- **Bidirecional**: transmite e recebe dados simultaneamente.
- Suporta **conexão de banda larga** — coexistência de múltiplas frequências no mesmo cabo.



Cabos de Fibra Óptica

- Feitos de **vidro**.
- Transmissão em **alta velocidade** e **longa distância**.
- **Taxa de erro muito baixa**, dispensando regeneração do sinal.
- Imunes a **ruídos eletromagnéticos**.



Transmissão via Rádio

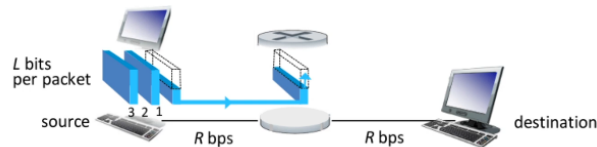
- Sinal transmitido no **espectro eletromagnético**.
- Comunicação **sem fio** e geralmente em modo **broadcast** (difusão).
- Operação **half-duplex**: alterna entre receber e enviar informações.
- **Problemas comuns de transmissão** :
 - ↪ Reflexão
 - ↪ Obstrução por objetos
 - ↪ Interferência
- **Tipos de transmissão via rádio** :
 - ↪ Micro-ondas terrestres
 - ↪ **Wireless LAN** (WiFi)
 - ↪ **Sem fio de longo alcance** (4G/5G)
 - ↪ **Satélite** — possui retardo de transmissão elevado.

1.4 Núcleo da Rede

- Parte **central da rede**, responsável por interconectar as redes de acesso.
- É composta por uma **malha de roteadores interconectados** que realizam o **encaminhamento de pacotes de dados** gerados pelos sistemas finais.

Comutação de Pacotes (*Packet Switching*)

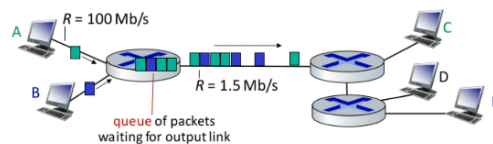
- O sistema final divide a **mensagem da aplicação** em **pacotes**.
- Esses pacotes são encaminhados roteador a roteador, seguindo um caminho até o destino.
- Cada pacote é transmitido utilizando **toda a capacidade do enlace** entre os roteadores.



- Cada pacote passa por uma série de roteadores desde a origem até o destino.
- Em cada roteador é necessário que o pacote seja **recebido completamente** antes de ser retransmitido.
- Delay fim-a-fim: $2L/R$ (assumindo delay de propagação nulo).
- **Exemplo:** $L = 10$ Kbits, $R = 100$ Mbps $\Rightarrow$ cada salto de transmissão leva 0.1 ms.

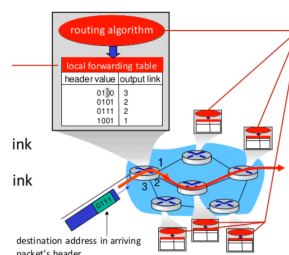
Atrasos e Perdas (*Queueing Delay, Loss*)

- Quando a **taxa de chegada** de pacotes em um roteador excede a **taxa de saída**, ocorre o enfileiramento.
- Pacotes são **enfileirados** aguardando o enlace ficar livre — adicionando retardo ao processo de transmissão fim a fim.
- Caso a fila de saída esteja cheia, os pacotes são **descartados (perdidos)**.



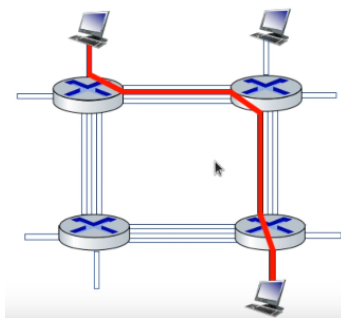
Funções Principais do Núcleo da Rede

- **Encaminhamento (Forwarding):** ação local — move pacotes do enlace de entrada para o enlace de saída apropriado.
- **Roteamento (Routing):** ação global — define os caminhos de origem a destino dentro da rede, utilizando **algoritmos de roteamento**.



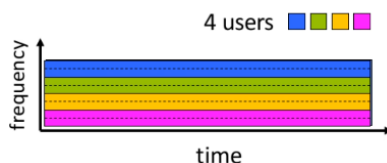
Alternativa: Comutação por Circuito

- Na comutação por circuito, existe uma **etapa inicial de conexão** onde :
 - ↪ É definida a rota completa do início ao fim.
 - ↪ São **alocados recursos** nos roteadores ao longo do caminho.
 - ↪ Cada roteador envolvido armazena informação sobre essa rota.
- Os pacotes transportam a informação sobre o **circuito virtual** ao qual pertencem.

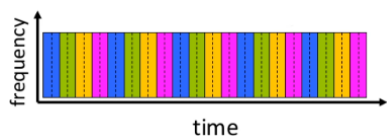


Técnicas de Multiplexação

- **Frequency Division Multiplexing (FDM) :**
 - ↪ Divide a capacidade do enlace em **faixas de frequência**.
 - ↪ Cada faixa é dedicada a um usuário, transmitindo até a taxa máxima da sua banda.



- **Time Division Multiplexing (TDM) :**
 - ↪ Divide o tempo de transmissão em **intervalos (slots)**.
 - ↪ Cada usuário transmite à taxa total do enlace durante o seu slot de tempo.



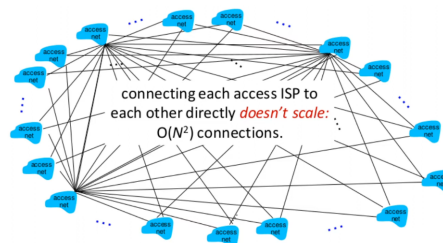
Comparação: Comutação de Pacotes vs. Comutação de Circuitos

- Ambas possuem a mesma **taxa de transmissão nominal**.
- A **comutação de pacotes** permite que mais usuários utilizem a rede simultaneamente.
- É mais eficiente para “**bursty data**” (rajadas de dados) :
 - ↪ Melhor **compartilhamento de recursos**.
 - ↪ Operação mais **simples**, sem necessidade de estabelecimento de chamada.

- **Problema** : Ausência de controle de congestionamento.
 - ⇒ Pode causar aumento de retardo e perda de pacotes devido ao transbordo das filas.
 - ⇒ Protocolos específicos são utilizados para controle de congestionamento.
- Aplicações como **vídeo/áudio streaming** ainda utilizam comutação de circuitos.

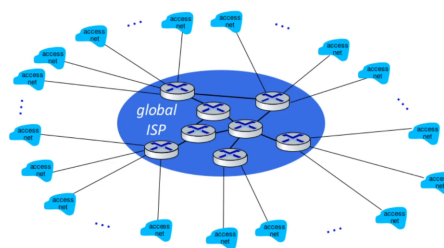
Estrutura da Internet

- Os **hosts** se conectam à Internet por meio de **provedores de acesso (ISPs)** :
 - ⇒ Domésticos, empresariais ou institucionais.
- Esses provedores precisam se **interconectar** para permitir comunicação entre seus clientes.
- Isso resultou em uma rede de **alta complexidade**, moldada por **decisões econômicas e políticas**.

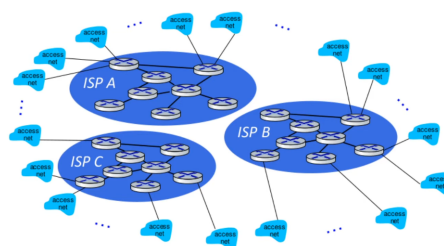


Hierarquização da Conexão

- Solução: criar um **provedor de acesso global**.
- Provedores locais se conectam a ele, estabelecendo uma relação **econômica** (pagamento pelo uso do serviço de trânsito).

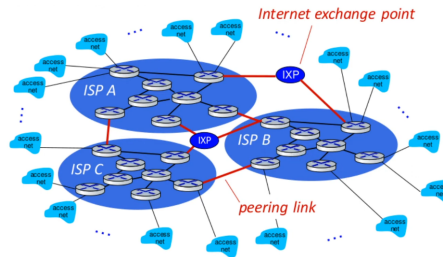


- Com o tempo, surgiu **concorrência** tanto por questões econômicas quanto geográficas.



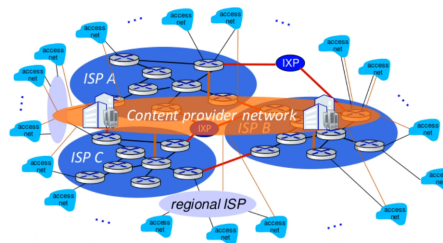
Peering e Pontos de Troca de Tráfego

- A conexão entre provedores locais ocorre por meio de :
 - ↪ **Peering físico (peering link)**
 - ↪ **Pontos de Troca de Tráfego (IXP)**

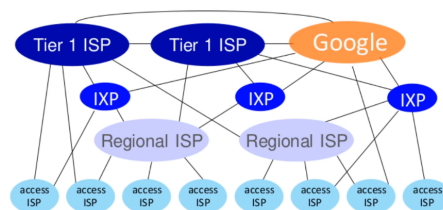


Provedores Regionais e Globais

- Devido à distância geográfica entre provedores globais e locais, surgiram **provedores regionais**.
- Grandes provedores de conteúdo (**Google, Microsoft, etc.**) se conectam diretamente aos provedores globais.

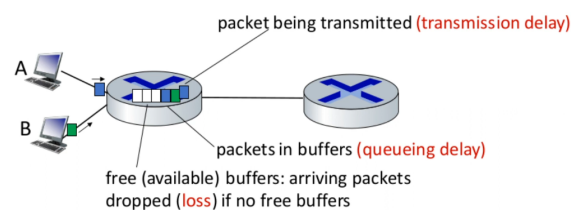


Hierarquia da Internet



1.5 Performance

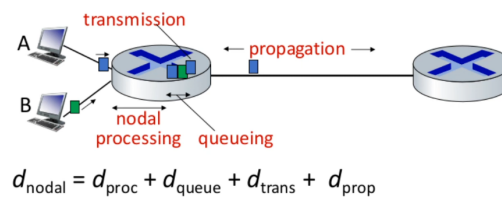
Como ocorre a perda e o retardo de pacotes ?



- Pacotes enfileirados esperam a sua vez de serem transmitidos.
- **Retardo de enfileiramento:** tempo que um pacote aguarda na fila para ser transmitido.
- **Retardo de transmissão:** tempo que leva para transmitir os bits no enlace.
- Se a **taxa de chegada** no enlace (temporariamente) excede a capacidade de saída, ocorre a **perda de pacotes**.

Retardo do Pacote: 4 Causas

- O retardo ocorre a cada nó (ou salto) na rede :



Componentes do Retardo :

- d_{proc} : **Retardo de Processamento do Nó**
 - ↪ Tempo gasto pelo roteador para processar o pacote que acabou de chegar.
 - ↪ Envolve tarefas como: Conferir a integridade do pacote e verificar o endereço de saída do pacote.
 - ↪ Geralmente menor que 1 ms.
- d_{queue} : **Retardo de Enfileiramento**
 - ↪ Tempo que o pacote aguarda na fila antes da transmissão.
 - ↪ Depende do **nível de congestionamento** no roteador.
- d_{trans} : **Retardo de Transmissão**
 - ↪ Tempo necessário para transmitir um pacote no enlace.
 - ↪ Dados : L = tamanho do pacote (bits), R = taxa de transmissão (bps)
 - ↪ Fórmula :
$$d_{\text{trans}} = \frac{L}{R}$$
- d_{prop} : **Retardo de Propagação**
 - ↪ Tempo necessário para que os bits do pacote propaguem até o destino.
 - ↪ d = comprimento do enlace, s = velocidade de propagação $\approx 2 \times 10^8$ m/s
 - ↪ Fórmula :
$$d_{\text{prop}} = \frac{d}{s}$$

Exemplo: Caravanas de Carros



O pedágio leva 12 s para atender cada carro (tempo de transmissão). A estrada tem pedágios a cada 100 km. Um carro se propaga a 100 km/h.

Analogia : 1 Carro $\rightarrow$ 1 bit, Caravana $\rightarrow$ pacote, Pedágio $\rightarrow$ roteador, Estrada $\rightarrow$ enlace.

Pergunta 1: Quanto tempo levaria para a caravana de 10 carros chegar ao 2º pedágio ?

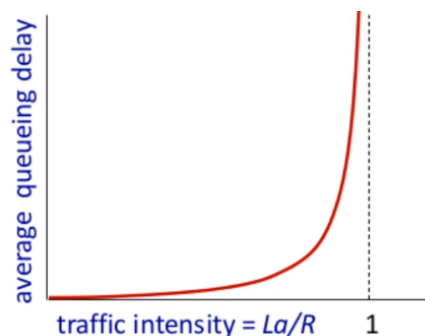
- Tempo de transmissão: $12 \times 10 = 120$ s.
- Tempo de propagação: $100 \text{ km} / 100 \text{ km/h} = 1$ h.
- Tempo total: **62 minutos**.

Pergunta 2: Se os carros se propagam a 1000 km/h e o pedágio leva 1 min por carro?

- Tempo de propagação: 6 min.
- O primeiro carro chega ao segundo pedágio após 7 min, antes de todos terminarem no primeiro pedágio.

Revisão: Retardo de Enfileiramento

- R = Capacidade de transmissão (bps)
- L = Tamanho do pacote (bits)
- a = Taxa de chegada (pacotes/s)
 - $\hookrightarrow \frac{La}{R} \approx 0$: Retardo pequeno.
 - $\hookrightarrow \frac{La}{R} \rightarrow 1$: Retardo grande.
 - $\hookrightarrow \frac{La}{R} > 1$: taxa de chegada maior que a capacidade de transmissão, retardo tende ao infinito.



Exemplo Real : traceroute

- $\hookrightarrow$ Programa que permite avaliar cada roteador ao longo da rota seguida pelos pacotes.
- $\hookrightarrow$ Mostra o **tempo de ida e volta (RTT)** de cada salto.

traceroute: gaia.cs.umass.edu to www.eurecom.fr

```

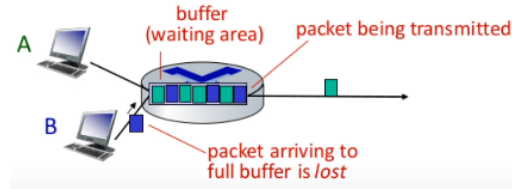
1 cs-gw (128.119.240.254) 1 ms 1 ms 2 ms
2 border1-rt-fa5-1-0.gw.umass.edu (128.119.3.145) 1 ms 1 ms 2 ms
3 cht-vbns.gw.umass.edu (128.119.3.130) 6 ms 5 ms 5 ms
4 in1-at1-0-0-19.wor.vbns.net (204.147.132.129) 16 ms 11 ms 13 ms
5 in1-so7-0-0-0.wae.vbns.net (204.147.136.136) 21 ms 18 ms 18 ms
6 abilene-vbns.abilene.ucaid.edu (198.32.11.9) 22 ms 18 ms 22 ms
7 nycm-wash.abilene.ucaid.edu (198.32.8.46) 22 ms 22 ms 22 ms
8 62.40.103.253 (62.40.103.253) 104 ms 109 ms 106 ms
9 de2-1.de1.de.geant.net (62.40.96.129) 109 ms 102 ms 104 ms
10 de.fr1.fr.geant.net (62.40.96.50) 113 ms 121 ms 114 ms
11 renater-gw.fr1.fr.geant.net (62.40.103.54) 112 ms 114 ms 112 ms
12 nio-n2.cssi.renater.fr (193.51.206.13) 111 ms 114 ms 116 ms
13 nice.cssi.renater.fr (195.220.98.102) 123 ms 125 ms 124 ms
14 r3t2-nice.cssi.renater.fr (195.220.98.110) 126 ms 126 ms 124 ms
15 eurecom-valbonne.r3t2.ft.net (193.48.50.54) 135 ms 128 ms 133 ms
16 194.214.211.25 (194.214.211.25) 126 ms 128 ms 126 ms
17 ***
18 ***
19 fantasia.eurecom.fr (193.55.113.142) 132 ms 128 ms 136 ms
  
```

Annotations:

- 3 delay measurements from gaia.cs.umass.edu to cs-gw.cs.umass.edu
- 3 delay measurements to border1-rt-fa5-1-0.gw.umass.edu
- trans-oceanic link
- looks like delays decrease! Why?
- * means no response (probe lost, router not replying)

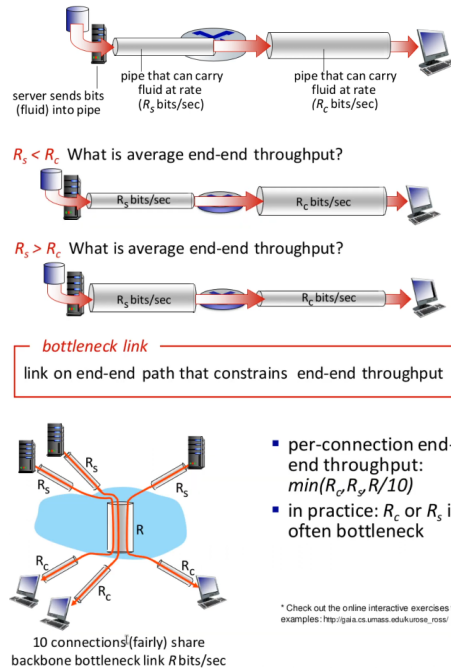
• Perda de Pacotes

- ⇒ A fila (**buffer**) precede um enlace de saída.
- ⇒ Quando a fila está cheia, o pacote é descartado.
- ⇒ Pacotes perdidos precisam ser **retransmitidos**.



• Vazão (Throughput)

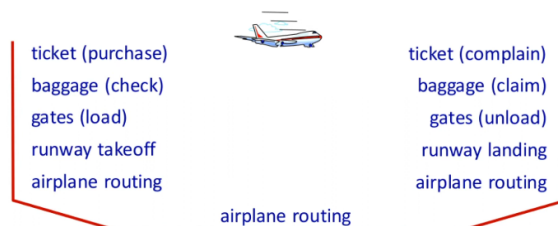
- ⇒ **Throughput**: taxa de transmissão efetiva percebida por uma conexão.
- ⇒ Tipos:
 - **Instantânea**: medida em um determinado instante.
 - **Média**: média ao longo de um período.
- ⇒ A vazão é sempre limitada pelo **link gargalo**.



1.6 Camadas de Protocolos e Modelos de Serviços

Questão inicial: Existia alguma maneira de organizar a estrutura da Internet?

Exemplo: Organização para Viajar de Avião



- Uma viagem de avião envolve uma série de etapas e serviços distintos.
- Cada etapa pode ser vista como uma **camada**, responsável por uma parte específica do processo.

Camadas :

- Cada camada implementa um **serviço** específico.
- Cada camada tem uma **contrapartida** tanto na origem quanto no destino.

ticket (purchase)	<i>ticketing service</i>	ticket (complain)
baggage (check)	<i>baggage service</i>	baggage (claim)
gates (load)	<i>gate service</i>	gates (unload)
runway takeoff	<i>runway service</i>	runway landing
airplane routing	<i>routing service</i>	airplane routing

Por que usar camadas ?

- A ideia surge naturalmente em sistemas complexos.
- **Abstrai** partes do sistema em diferentes camadas.
- Cada camada deve ser **bem definida** e possuir uma **interface clara**.
- Facilita a **modularização**, **manutenção** e **atualização** do sistema.
- Problema: pode haver **redundância de ações** entre camadas.

Pilha de Protocolos da Internet (TCP/IP)

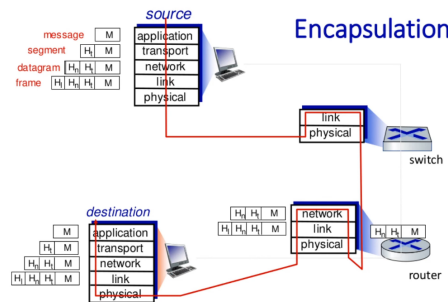
- **Aplicação** : Suporta as aplicações que estão executando nos sistemas finais.
⇒ Exemplos: HTTP, SMTP, IMAP.
- **Transporte** : Responsável pela **transferência de pacotes entre processos** que estão executando nos sistemas finais.
⇒ Exemplos: UDP, TCP.
- **Rede** : Faz o **roteamento dos pacotes** da origem até o destino.
⇒ Exemplos: IP, Protocolos de roteamento.
- **Enlace** : Garante a **transferência de dados entre dispositivos diretamente conectados**.
⇒ Exemplos: Ethernet, 802.11 (WiFi), PPP.
- **Física** : Responsável pela **transmissão dos bits** através do meio físico.

Exemplo de camadas de protocolos :

Aplicação
Transporte
Rede
Enlace
Física

Encapsulamento :

- Cada camada adiciona seu próprio cabeçalho à unidade de dados recebida da camada superior.
- O processo é **reverso** no destino: cada camada remove seu cabeçalho correspondente.
- Esse mecanismo garante que cada camada só interaja com sua correspondente no outro sistema.



2 Camada de Aplicação

2.1 Princípios da Camada de Rede

Objetivos

- Compreender aspectos conceituais e de implementação dos protocolos da camada de aplicação
 - ↪ Modelos de serviço da camada de transporte
 - ↪ Paradigma **cliente-servidor**
 - ↪ Paradigma **par a par** (peer-to-peer)
- Aprender sobre protocolos por meio da análise dos protocolos populares da camada de aplicação e sua infraestrutura:
 - ↪ HTTP
 - ↪ SMTP, IMAP
 - ↪ DNS
 - ↪ Sistemas de streaming de vídeo, CDNs
- Programação de aplicações de rede:
 - ↪ API de sockets

Exemplos de Aplicativos de Rede

- Redes Sociais
- Internet
- Mensagens de Texto
- E-mail
- Jogos online com vários usuários
- Transmissão de vídeo armazenado (YouTube, Hulu, Netflix)
- Compartilhamento de arquivos P2P
- Chamada sobre IP
- Videoconferência em tempo real (por exemplo, Zoom)
- Busca na Internet
- Acesso remoto

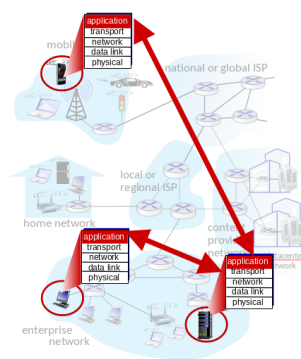
Criando um App de Rede

Escrevemos programas que:

- Executem em diferentes sistemas finais.
- Se comuniquem pela rede.
- Exemplo: O software de servidor web se comunica com o software de navegador.

Não há necessidade de escrever software para dispositivos do núcleo da rede

- Dispositivos do núcleo da rede não executam aplicações de usuário.
- Aplicações em sistemas finais permitem o rápido desenvolvimento e a propagação de aplicativos.



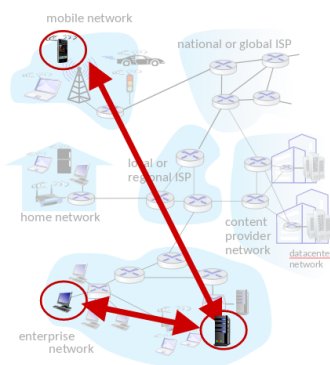
Paradigma Cliente-Servidor

Servidor:

- Host sempre ativo.
- Endereço IP permanente.
- Frequentemente em data centers, para fins de escalabilidade.

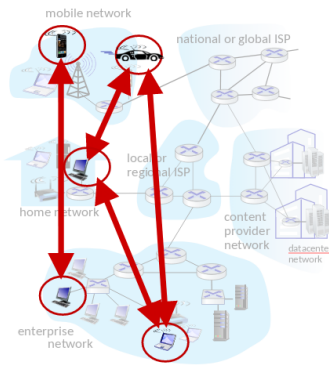
Cliente:

- Contatar, comunicar-se com o servidor.
- Podem estar conectados de forma intermitente.
- Podem ter endereços IP dinâmicos.
- Não se comunicam diretamente entre si.
- Exemplos: HTTP, IMAP, FTP.



Arquitetura Par a Par

- Sem servidor sempre ativo.
- Sistemas finais arbitrários comunicam-se diretamente.
- Pares solicitam serviços a outros pares e, em contrapartida, prestam serviços a outros pares.
 - ↪ Escalabilidade Intrínseca: Novos pares trazem nova capacidade de serviço, bem como novas demandas de serviço.
- Pares conectam-se de forma intermitente e alteram seus endereços IP.
 - ↪ Gerenciamento complexo ...
- Exemplo: Compartilhamento de arquivos P2P [BitTorrent]

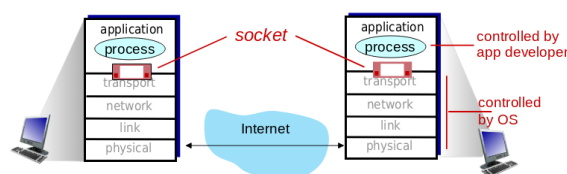


Processos de Comunicação

- **Processo**: programa em execução em um host.
- No mesmo host, dois processos comunicam-se utilizando **comunicação entre processos** (definida pelo SO).
- Processos em hosts diferentes comunicam-se por meio da troca de **mensagens**.
- Nota: Aplicações com arquiteturas P2P possuem processos clientes e processos servidores.
 - ↪ **Processo cliente**: Processo que inicia a comunicação.
 - ↪ **Processo servidor**: Processo que aguarda ser contatado.

Sockets

- O processo envia/recebe mensagens para/de seu **socket**.
- o socket é análogo a uma porta:
 - ↪ O processo remetente empurra a mensagem para fora pela porta.
 - ↪ O processo remetente conta com a infraestrutura de transporte do outro lado da porta para entregar a mensagem ao socket do processo receptor.
 - ↪ Dois sockets envolvidos: um de cada lado.



Processos de Endereçamento

- Para receber mensagens, o processo precisa de um **identificador**.
- O dispositivo hospedeiro possui um endereço IP de 32 bits exclusivo.
- O endereço IP do host onde o processo é executado é não suficiente para identificar o processo pois vários processos podem estar em execução no mesmo host.
- O **identificador** inclui tanto o **endereço IP** quanto os **números de porta** associados ao processo no hospedeiro.
- Exemplos de números de porta:
 - ↪ Servidor HTTP: 80
 - ↪ Servidor de email: 25

Um protocolo da camada de aplicação define:

- **Tipos de mensagens** trocadas.
 - ↪ Ex: request (requisição), response (resposta).
- **Sintaxe da Mensagem:** Quais campos compõem as mensagens e como eles são delimitados.
- **Semântica da Mensagem:** Significado das informações nos campos.
- **Regras** sobre quando e como os processos enviam mensagens e respondem a elas.
- **Protocolos Abertos:**
 - ↪ Definidos em RFCs; todos têm acesso à definição do protocolo.
 - ↪ Permitem a interoperabilidade.
 - ↪ Exemplo: HTTP, SMTP.
- Protocolos Proprietários:
 - ↪ Exemplo: Zoom.

Serviços de Transporte que um aplicativo/app precisa

- **Integridade de Dados:**
 - ↪ Algumas aplicações (por exemplo, transferência de arquivos, transações web) exigem transferência de dados 100% confiável.
 - ↪ Outras aplicações (por exemplo, áudio) podem tolerar alguma perda.
- **Timing/Temporização:**
 - ↪ Alguns aplicativos (por exemplo, telefonia via Internet, jogos interativos) exigem baixa latência para serem “eficazes”.
- **Throughput/Vazão**
 - ↪ Alguns aplicativos (por exemplo, multimídia) exigem uma taxa de transferência mínima para serem “eficazes”.
 - ↪ Outros aplicativos (“aplicativos elásticos”) utilizam qualquer taxa de transferência que lhes seja disponibilizada.
- **Security/Segurança**
 - ↪ Criptografia, Integridade dos dados ...

Requisitos de Serviços de Transporte: Aplicativos Comuns

Aplicação	Perda	Taxa	Sensível ao atraso?
Transf./download de arquivos	Sem perda	Elástica	Não
E-mail	Sem perda	Elástica	Não
Documentos Web	Sem perda	Elástica	Não
Áudio/vídeo em tempo real	Tol. à perda	Áudio: 5 Kbps–1 Mbps Vídeo: 10 Kbps–5 Mbps	Sim, dezenas de ms
Áudio/vídeo por streaming	Tol. à perda	Mesmo que acima	Sim, alguns segundos
Jogos interativos	Tol. à perda	Kbps+	Sim, dezenas de ms
Mensagens de texto	Sem perda	Elástica	Sim e não

Serviços de Protocolos de Transporte da Internet

Serviço TCP	Serviço UDP
• Transporte confiável entre os processos de envio e recepção; • Controle de fluxo: O emissor não sobrecarrega o receptor; • Controle de congestionamento: Reduz a taxa de envio quando a rede está congestionada; • Orientado à conexão: Requer estabelecimento de conexão entre cliente e servidor; • Não fornece: Garantias de tempo, vazão mínima ou segurança.	• Transferência de dados não confiável entre os processos de envio e recepção; • Não fornece: Confiabilidade, controle de fluxo, controle de congestionamento, garantias de tempo, vazão mínima, segurança ou estabelecimento de conexão.

Exemplo: Aplicações de Internet e Protocolos de Transporte

Aplicação	Protocolo da aplicação	Protocolo de transporte
Transferência/download de arquivos	FTP (RFC 959)	TCP
E-mail	SMTP (RFC 5321)	TCP
Documentos Web	HTTP (RFC 7230, RFC 9110)	TCP
Telefonia pela Internet	SIP (RFC 3261), RTP (RFC 3550) ou proprietário	TCP ou UDP
Áudio/vídeo por streaming	HTTP (RFC 7230), DASH	TCP
Jogos interativos	WOW, FPS (proprietário)	UDP ou TCP

Segurança no TCP

- Vanilla TCP e UDP sockets:
 - ↔ Sem criptografia.
 - ↔ Senhas em texto simples/claro enviadas pelo socket trafegam pela Internet em texto simples/claro (!)
- Transport Layer Security (TLS):
 - ↔ Fornece conexões TCP criptografadas.
 - ↔ Integridade de dados.
 - ↔ Autenticação de pontos finais.

- TLS é implementado na camada de aplicação.
 - ↪ Aplicações utilizam bibliotecas TLS, que por sua vez utilizam TCP.
 - ↪ Dados em texto simples/claro enviados para o “socket” atravessam a Internet criptografados.

2.2 Web e HTTP

- Uma página da Web consiste em **objetos**, cada um dos quais pode estar armazenado em servidores Web diferentes.
- Um objeto pode ser um arquivo HTML, uma imagem JPEG, um applet Java, um arquivo de áudio, ...
- Uma página da Web consiste em um **arquivo HTML base** que inclui **vários objetos referenciados**, cada um endereçável por uma **URL**.

www.someschool.edu / someDept/pic.gif
Host name Path name

HTTP: hypertext transfer protocol

- Protocolo Web da camada de aplicação.
- Modelo cliente/servidor:
 - ↪ **Cliente:** Navegador que solicita (*requests*), recebe (usando o protocolo HTTP) e “exibe” objetos da Web.
 - ↪ **Servidor:** Servidor Web que envia (usando o protocolo HTTP) objetos em resposta (*response*) a solicitações (*requests*).



O HTTP utiliza TCP:

- O cliente inicia uma conexão TCP (cria um socket) com o servidor, na porta 80.
- O servidor aceita a conexão TCP do cliente.
- Mensagens HTTP (mensagens do protocolo da camada de aplicação) são trocadas entre o navegador (cliente HTTP) e o servidor Web (servidor HTTP).
- A conexão TCP é encerrada.

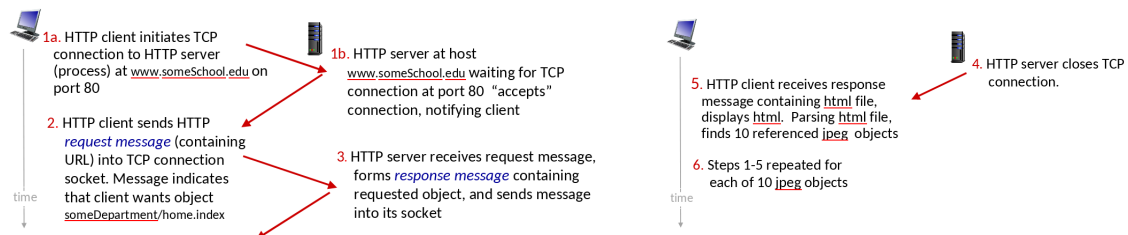
O HTTP é “sem estado” (*stateless*)

- O servidor não mantém informações sobre solicitações anteriores do cliente.
- **Obs: Protocolos que mantêm o "estado" são complexos!**
 - ↪ O histórico anterior (estado) precisa ser mantido.
 - ↪ Se o servidor ou o cliente falhar, suas visões do “estado” podem ficar inconsistentes e precisam ser reconciliadas.

Conexões HTTP: Dois Tipos

- **HTTP não persistente (*Non-persistent HTTP*)**
 - ↪ Conexão TCP aberta.
 - ↪ No máximo um objeto enviado pela conexão TCP.
 - ↪ Conexão TCP fechada.
 - ↪ **Obs:** O download de múltiplos objetos exigia múltiplas conexões.
- **HTTP Persistente (*Persistent HTTP*)**
 - ↪ Conexão TCP aberta com um servidor.
 - ↪ Múltiplos objetos podem ser enviados por meio de uma única conexão TCP entre o cliente e esse servidor.
 - ↪ Conexão TCP encerrada.

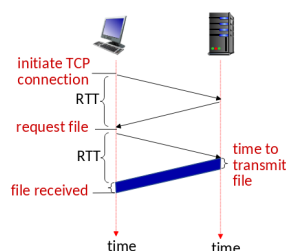
HTTP não persistente: Exemplo



HTTP não persistente: Tempo de Resposta

- **RTT:** Tempo para um pacote pequeno viajar do cliente ao servidor e retornar.
- **Tempo de resposta HTTP/*HTTP response time* (por objeto):**
 - ↪ Um RTT para iniciar a conexão TCP.
 - ↪ Um RTT para a requisição HTTP e o retorno dos primeiros bytes da resposta HTTP.
 - ↪ Tempo de transmissão do objeto/arquivo.

Tempo de resposta do HTTP não persistente = 2 RTT + tempo de transmissão do arquivo



HTTP Persistente (HTTP 1.1)

- **Problemas do HTTP não persistente:**
 - ↪ Requer 2 RTTs por objeto.
 - ↪ Sobrecarga do SO para cada conexão TCP.
 - ↪ Navegadores frequentemente abrem múltiplas conexões TCP paralelas para buscar objetos referenciados em paralelo.
- **HTTP Persistente (HTTP 1.1):**
 - ↪ O servidor mantém a conexão aberta após enviar a resposta.
 - ↪ Mensagens HTTP subsequentes entre o mesmo cliente e servidor são enviadas pela conexão aberta.
 - ↪ O cliente envia requisições assim que encontra um objeto referenciado.
 - ↪ Apenas um RTT para todos os objetos referenciados (reduzindo o tempo de resposta pela metade).

Mensagem de *request*(solicitação) HTTP

- Dois tipos de mensagens HTTP: *request*(*requisição*), *response*(*resposta*).
- **HTTP request message: ASCII**
 - ↪ Formato legível por humanos.

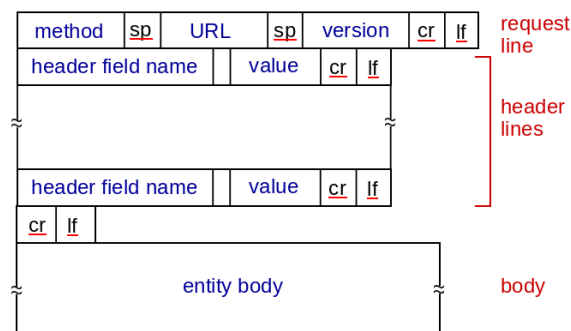
request line (GET, POST, HEAD commands) → GET /index.html HTTP/1.1\r\n

header lines → Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:80.0) Gecko/20100101 Firefox/80.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
\r\n

carriage return character
line-feed character

carriage return, line feed at start of line indicates end of header lines

Formato General da Mensagem



Outras *request messages* (mensagens de solicitação) HTTP

- **Método POST:**
 - ↪ Uma página da web frequentemente inclui campos de formulário.
 - ↪ A entrada do usuário é enviada do cliente para o servidor no corpo da entidade de uma mensagem de requisição HTTP POST.

- **Método GET:** Para enviar dados ao servidor
 - ↪ Incluir dados do usuário no campo URL da mensagem de requisição HTTP GET (após um '?'):
 - ↪ `www.somesite.com/animalsearch?monkeys&banana`
- **Método HEAD:**
 - ↪ Solicita apenas os cabeçalhos (*headers*) que seriam retornados se a URL especificada fosse requisitada com o método HTTP GET.
- **Método PUT:**
 - ↪ Faz o upload de um novo arquivo (objeto) para o servidor.
 - ↪ Substitui completamente o arquivo existente na URL especificada pelo conteúdo do corpo da entidade da mensagem de requisição HTTP POST.

HTTP response message

```
status line (protocol status code status phrase) → HTTP/1.1 200 OK
                                                    Date: Tue, 08 Sep 2020 00:53:20 GMT
                                                    Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips
                                                    PHP/7.4.9 mod_perl/2.0.11 Perl/v5.16.3
                                                    Last-Modified: Tue, 01 Mar 2016 18:57:50 GMT
                                                    ETag: "a5b-52d015789ee9e"
                                                    Accept-Ranges: bytes
                                                    Content-Length: 2651
                                                    Content-Type: text/html; charset=UTF-8
                                                    \r\n
data, e.g., requested HTML file → data data data data data ...
```

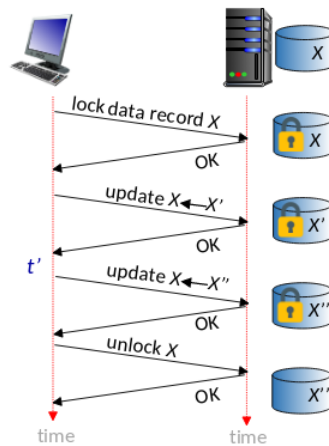
HTTP response status codes

- O código de status aparece na primeira linha da mensagem de resposta do servidor para o cliente.
- **200 OK**
 - ↪ Solicitação/requisição bem-sucedida; objeto solicitado mais adiante nesta mensagem.
- **301 Moved Permanently**
 - ↪ O objeto solicitado foi movido; a nova localização está especificada mais adiante nesta mensagem (no campo *Location*:).
- **400 Bad Request**
 - ↪ Mensagem de solicitação (*request message*) não compreendida pelo servidor.
- **404 Not Found**
 - ↪ Documento solicitado não encontrado neste servidor.
- **505 HTTP Version Not Supported**

Manutenção do estado do usuário/servidor: cookies

- Lembre-se: a interação HTTP GET/resposta é **sem estado**.
- Não há noção de trocas de mensagens HTTP em várias etapas para concluir uma “transação” na Web.
 - ↪ Não há necessidade de o cliente/servidor rastrear o “estado” de uma troca em várias etapas.
 - ↪ Todas as solicitações HTTP são independentes umas das outras.
 - ↪ Não há necessidade de o cliente/servidor “se recuperar” de uma transação parcialmente concluída, mas nunca totalmente finalizada.

- Um protocolo *stateful* (com estado): o cliente faz duas alterações em X, ou nenhuma.



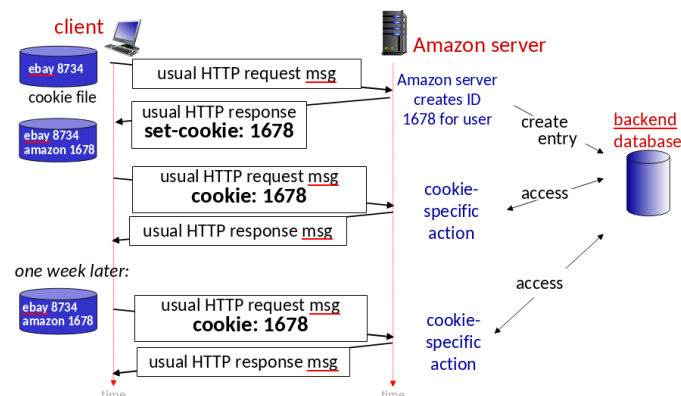
Sites e navegadores de clientes utilizam **cookies** para manter um estado entre transações.

Quatro Componentes:

1. Linha de cabeçalho de cookie da mensagem de resposta HTTP (*HTTP response message*).
2. Linha de cabeçalho de cookie na próxima mensagem de requisição HTTP (*HTTP request message*).
3. Arquivo de cookie armazenado no host do usuário, gerenciado pelo navegador do usuário.
4. Banco de dados de back-end no site

Exemplo:

- Susan usa o navegador em seu notebook e acessa um site de e-commerce específico pela primeira vez.
- Quando as requisições HTTP iniciais chegam ao site, este cria:
 - ↪ Um ID exclusivo (também conhecido como “cookie”).
 - ↪ Uma entrada no banco de dados de backend para esse ID.
- As requisições HTTP subsequentes de Susan para esse site conterão o valor do ID do cookie, permitindo que o site “identifique” Susan.



HTTP cookies: Comentários

- Em que os cookies podem ser usados ?
 - ↪ Autorização
 - ↪ Carrinhos de Compra
 - ↪ Recomendações
 - ↪ Estado da sessão do usuário (Webmail)
- Desafio: Como manter o estado ?
 - ↪ **Nos *endpoints* do protocolo:** Manter o estado no Emissor/receptor ao longo de múltiplas transações.
 - ↪ **Nas mensagens:** cookies em mensagens HTTP carregam o estado.
- **cookies e privacidade:**
 - ↪ Os cookies permitem que os sites obtenham muitas informações sobre você durante a sua visita.
 - ↪ cookies persistentes de terceiros (cookies de rastreamento) permitem que uma identidade comum (valor do cookie) seja rastreada em vários sites.

2.3 E-mail, SMTP, IMAP

2.4 DNS: Domain Name System

2.5 Streaming de Vídeo e Distribuição de Conteúdo na Internet

2.6 Programação de Socket com UDP e TCP

dasd

- dasd
 - ↪ po